Week 12 Lab Handout

Monitoring — Logs and Numbers, So You Can See Your API

Time: 110 minutes

Bring: your laptop, your API from Weeks 5–11 (with auth and rate limiting), and your ring partner. They will send traffic to your API today

---

Before you start: you can't fix what you can't see

> Your API is live now, and nobody is watching every request. Last week your feature coped with a partner that failed, but how would you ever know it happened, or how often? Right now the answer is: only if you were staring at the terminal at that exact moment.

Two tools fix that. **Logs** say *what happened* — one line per request. **Metrics** say *how much* — counts and averages across all requests. Together they make your API **observable**: you can tell what it is doing from the outside, without attaching a debugger or guessing.

Today you build the smallest version of both, send real traffic through them, and read what your own API tells you.

Part A — Decide What to Record, and Make a Prediction (10 min)

Logs are only useful if they hold the right things, and dangerous if they hold the wrong ones. A log file gets copied, shared in chats, pasted into bug reports, and committed by accident. Decide the rule before you write any code:

Log this	Never log this
Timestamp	API keys or tokens, even partly
Method and path	Passwords
Status code	Full request or response bodies
Response time in milliseconds	Query strings (they can carry emails, tokens, names)
A label for *who* called	Anything the caller would be upset to see in a screenshot

"Who called" is useful: it separates your tests from your partner's traffic. But you must do it **without writing the key**. Today you'll log a short fingerprint of the key instead.

Now make a prediction, before you have any data. Create `MONITORING_NOTES.md` in your repo root and write down:

1. Which of your endpoints do you think will be the **slowest**, and why?
2. Which do you think will produce the most **errors**, and why?

You'll compare these to the real numbers in Part E.

Part B — Add Request Logging (30 min)

Create `monitoring.js` with this middleware. It starts a timer when a request arrives, and when the response has been sent it writes one line of JSON:

```
const crypto = require('crypto');

const round = n => Math.round(n * 10) / 10;

// A label for who called, without ever writing the key itself.
function who(req) {
  if (!req.apiKey) return 'none';
  const hash = crypto.createHash('sha256').update(req.apiKey).digest('hex');
  return 'key-' + hash.slice(0, 6);
}

function monitor(req, res, next) {
  const start = process.hrtime.bigint();

  res.on('finish', () => { // after the response is sent
    const ms = Number(process.hrtime.bigint() - start) / 1e6;

    console.log(JSON.stringify({
      time: new Date().toISOString(),
      method: req.method,
      path: req.path, // path only: no query string
      status: res.statusCode,
      ms: round(ms),
      caller: who(req),
    }));
  });

  next();
}

module.exports = { monitor };
```

What it does, piece by piece:

- `res.on('finish', ...)` runs *after* the response has gone out, so the status code and the timing are final. A timer started at the top and read here measures the whole request.
- `process.hrtime.bigint()` is a precise clock. Using it avoids the 0 ms readings you get from rounding whole milliseconds.
- `req.path` is the path only. `req.originalUrl` would include the query string, so never use it in a log.
- `who(req)` hashes the key and keeps six characters. The same key always gives the same label, so you can tell callers apart, and nobody can recover the key from it.
- **One JSON object per line** is easy to read, easy to `grep`, and easy to count with a script.

Now wire it in. **Order matters again**, just like in Week 9:

```
const { monitor } = require('./monitoring');

app.use(monitor); // FIRST, so rejected requests are logged too
```

```
app.use(requireApiKey);
app.use(rateLimit({ windowMs: 60000, max: 100 }));
// ...your routes
```

If you register `monitor` *after* the key check, every request that gets a 401 is rejected before the logger ever runs, and those are exactly the requests you most want to see.

Steps:

1. Add `monitoring.js` and wire it in as above.
2. Restart your server with its output copied to a file, and add `*.log` to `.gitignore`:

```
node server.js | tee requests.log
```

3. Send three requests from another terminal: one with your key, one with a wrong key, one with no key.

```
curl -s -H "X-API-Key: <your-real-key>" http://localhost:3000/tasks
curl -s -H "X-API-Key: wrong" http://localhost:3000/tasks
curl -s http://localhost:3000/tasks
```

4. Look at the three log lines. You should see a 200 and two 401s, and `caller` should be a `key-xxxxxx` label for the real key and `none` for the others. For example:

```
{"time":"2026-10-28T05:27:39.185Z","method":"GET","path":"/tasks","status":200,"ms":5.2,
"caller":"key-dc87f9"}
{"time":"2026-10-28T05:27:39.298Z","method":"GET","path":"/tasks","status":401,"ms":1.1,
"caller":"none"}
```

5. **Prove the key isn't in the log.** This should print `0`:

```
grep -c "<your-real-key>" requests.log
```

Keep a few of these lines. They are part of your deliverable.

Part C — Add a /stats Endpoint (30 min)

Logs show individual requests. To see the whole picture you also need running totals: how many requests, how many failures, and how long they take. You'll keep them in memory and add an endpoint that shows them. This is not a monitoring product, just three small additions to the same `monitoring.js` file.

Addition 1. Add this after the `round` line near the top. It holds the totals, and builds an empty record for a new route:

```
const stats = {
  startedAt: new Date().toISOString(),
  total: 0,
  clientErrors: 0, // 4xx: usually the caller's mistake
  serverErrors: 0, // 5xx: our fault
  totalMs: 0,
  routes: {},
};

function newRoute() {
```

```
  return { count: 0, clientErrors: 0, serverErrors: 0, totalMs: 0, maxMs: 0 };
}
```

Addition 2. Inside the `res.on('finish', ...)` block, add this right after the `console.log(...)` statement, so that every finished request also updates the totals:

```
    // Group by route pattern: /tasks/1 and /tasks/2 both become GET /tasks/:id
    const route = req.route ? req.baseUrl + req.route.path : '(no route)';
    const label = `${req.method} ${route}`;
    const r = stats.routes[label] || newRoute();
    stats.routes[label] = r;

    stats.total += 1;
    stats.totalMs += ms;
    r.count += 1;
    r.totalMs += ms;
    r.maxMs = Math.max(r.maxMs, ms);
    if (res.statusCode >= 500) {
      stats.serverErrors += 1;
      r.serverErrors += 1;
    } else if (res.statusCode >= 400) {
      stats.clientErrors += 1;
      r.clientErrors += 1;
    }
```

Two details matter here. First, the route label uses `req.route.path`, which is the *pattern*, so `/tasks/1` and `/tasks/2` both become `GET /tasks/:id`. If you used the raw path, every different id would become its own row and the list would grow forever. Second, errors are counted in two groups. A **4xx** usually means the caller made a mistake. A **5xx** means *you* did. Your error rate is built from the 5xx count only.

Addition 3. Add this function above the `module.exports` line, and change that line to export both functions:

```
function statsHandler(req, res) {
  const routes = {};
  for (const [label, r] of Object.entries(stats.routes)) {
    routes[label] = {
      count: r.count,
      clientErrors: r.clientErrors,
      serverErrors: r.serverErrors,
      avgMs: round(r.totalMs / r.count),
      maxMs: round(r.maxMs),
    };
  }
  const rate = stats.total ? (stats.serverErrors / stats.total) * 100 : 0;
  const avg = stats.total ? stats.totalMs / stats.total : 0;

  res.json({
    startedAt: stats.startedAt,
    uptimeSeconds: Math.round(process.uptime()),
    total: stats.total,
    clientErrors: stats.clientErrors,
    serverErrors: stats.serverErrors,
    errorRatePercent: round(rate),
    avgMs: round(avg),
    routes,
```

```
  });
}

module.exports = { monitor, statsHandler };
```

Finally, register the endpoint **after** your key check, so it needs a key like everything else:

```
const { monitor, statsHandler } = require('./monitoring');

app.use(monitor);
app.use(requireApiKey);
app.use(rateLimit({ windowMs: 60000, max: 100 }));
app.get('/stats', statsHandler);
// ...your routes
```

Restart, send a few requests, then read your numbers:

```
curl -s -H "X-API-Key: <your-real-key>" http://localhost:3000/stats
```

Reading the response:

- `total` is every request that finished since the server started.
- `clientErrors` **and** `serverErrors` are the 4xx and 5xx counts.
- `errorRatePercent` is server errors divided by total, as a percent.
- `avgMs` is the mean response time, and `maxMs` (per route) is the slowest single request.
- `routes` breaks all of it down per endpoint. The row called `(no route)` holds requests that never reached a route: a 401 or 429 that was rejected earlier, and a 404 for a path that doesn't exist. Check the log lines to tell those apart.

Honest limits. The numbers live in your server's memory, so they reset every time you restart, and they would be wrong across several server copies. That is fine for this course. Production systems send them to a shared tool. Also, `/stats` doesn't count itself in its own reply, but it appears in the next one.

Stretch, if you finish early: in your Week 11 `/upcoming` route, add a counter that goes up whenever the fallback serves saved data, and include it in `/stats`. It tells you how often your partner is really failing.

Part D — Generate Traffic and Watch (20 min)

Monitoring with no traffic shows you nothing. Save this as `traffic.js`. It sends a random mix of good requests, missing ids, requests with no key, and paths that don't exist. Edit the `mix` list so the paths match **your** API:

```
// traffic.js: sends a mix of requests to your API, to give you data.
// Run: API=http://localhost:3000 KEY=<your-key> node traffic.js
// Adapt the paths in `mix` to your own API.
const API = process.env.API || 'http://localhost:3000';
const KEY = process.env.KEY;
const N = Number(process.env.N || 60);

const mix = [
  { method: 'GET', path: '/tasks', key: true },
  { method: 'GET', path: '/tasks', key: true },
```

```
  { method: 'GET', path: '/tasks/1', key: true },
  { method: 'GET', path: '/tasks/99', key: true }, // no such id
  { method: 'GET', path: '/tasks', key: false }, // no key at all
  { method: 'GET', path: '/not-a-route', key: true }, // no such path
];

const pause = ms => new Promise(resolve => setTimeout(resolve, ms));

(async () => {
  const counts = {};
  for (let i = 0; i < N; i++) {
    const r = mix[Math.floor(Math.random() * mix.length)];
    try {
      const res = await fetch(API + r.path, {
        method: r.method,
        headers: r.key ? { 'X-API-Key': KEY } : {},
      });
      counts[res.status] = (counts[res.status] || 0) + 1;
    } catch (err) {
      counts.failed = (counts.failed || 0) + 1;
    }
    await pause(50);
  }
  console.log('Status codes received:', counts);
})();
```

1 Run it against your own API:

```
API=http://localhost:3000 KEY=<your-real-key> node traffic.js
```

Watch your server terminal scroll, then read the status summary the script prints.

2 **Get your ring partner involved.** Ask them to run the same script against *your* URL, with the key you issued them. Their requests should show up in your log with a *different* `caller` label from yours. If they don't, check that the key you gave them is the one in your `API_KEYS` list.

3 Run it a few more times, and change `N` (for example `N=200`) to send more. If the rate limit from Week 9 kicks in, you will see 429s. That is data too.

4 Read `/stats` again, and keep a copy of the output for your notes.

Don't invent a pattern. Your API's own traffic will show something. If everything looks flat and healthy, that is a valid finding: say so, with numbers. Then test one idea on purpose, for example load a few hundred rows into your database and see whether your list endpoint slows down.

Part E — Read Your Own Data (15 min)

Now answer questions with numbers from `/stats` and your log, not with guesses:

Question	Where to look
Which endpoint is slowest on average?	`avgMs` per route
Which one has a slow *worst case*?	`maxMs` per route. An average can hide one very slow request
Which endpoint fails most often?	`serverErrors` and `clientErrors` per route
Are the 401s a mistake or probing?	Log lines with `caller: "none"`, and who sent them
Is the error rate healthy?	`errorRatePercent`, and whether any 5xx appear at all

Now finish `MONITORING_NOTES.md`. Add to the predictions from Part A:

1. **What you predicted vs. what the data shows,** with the actual numbers.
2. **One real pattern** in your own traffic, stated with figures (for example "GET /tasks/:id averaged 4.1 ms but its slowest call was 38 ms, and 9 of its 16 calls were 404s").
3. **Why you think it happens,** and what you would check next to find out. A hypothesis is fine, and "I don't know yet, so I would look at..." is a good answer.
4. **One alert you would want in production.** Pick one number from `/stats`, choose a threshold, and say what should happen when it fires. For example, an error rate above a few percent for five minutes. Think back to Week 11: while a human is paged and fixes the cause, what should your feature already be doing for users?

Part F — Document and Push (5 min)

1. Add a short `/stats` section to your `README.md`: the endpoint, that it needs a key, and an example response (use `<your-api-key>` as the placeholder, never a real key).
2. Check that nothing sensitive is about to be committed:

```
grep -c "<your-real-key>" requests.log
git status
```

The first should print `0`. In `git status`, `.env` and `requests.log` must **not** be listed.

3. Commit and push:

```
git add . monitoring.js MONITORING_NOTES.md README.md
git commit -m "Week 12: request logging and /stats endpoint"
git push
```

This week's deliverable checklist

- Every request is logged as one line with time, method, path, status, and response time
- Rejected requests (401, 429) appear in the log, because `monitor` runs first

- The log never contains a key, a query string, or a request body, and `grep` for your key prints 0
- `/stats` works, needs a key, and reports total, 4xx and 5xx counts, error rate, average time, and a per-route breakdown
- Sample log output saved or screenshotted, including at least one 200 and one 401
- Traffic generated (your own, plus your partner's) and `/stats` output captured
- `MONITORING_NOTES.md`: predictions, a real pattern with numbers, a hypothesis, and one alert idea
- `/stats` documented in the README, and everything committed and pushed

Rubric (20 pts)

Criterion	Points
Logging captures the right fields, excludes sensitive data	6
Stats/health endpoint functional and accurate	8
Reflection shows genuine data-driven observation	6

Common problems and quick fixes

Problem	Fix
Rejected requests (401, 429) are missing from the log	`monitor` is registered after the key check or rate limiter. Move `app.use(monitor)` to the very top
The `ms` value is 0, or `NaN`	Take the end time inside the `finish` handler, with the same clock as the start. Don't mix `Date.now()` and `hrtime`
Every id shows up as its own row in `/stats`	You grouped by `req.path`. Use `req.route.path` so `/tasks/1` and `/tasks/2` share one row
`/stats` returns 401	It sits behind your key check, on purpose. Send your `X-API-Key` header
`/stats` shows more requests than you sent	Your partner's traffic, the traffic script, and the 401s all count. Use `caller` in the log to separate them
Your key appears in the log	Something is logging headers, the full URL, or the body. Remove it, replace the key, and if it was committed, treat it as leaked (Week 9)
The average looks fine but the API feels slow	An average hides outliers. Read `maxMs` per route and look at the slowest log lines
`(no route)` has a high count	It mixes 401s, 429s, and 404s. Read the log lines to see which is which

Next week: Versioning. Now that your log shows which callers use which endpoints, you can see exactly who a breaking change would hurt. That is the question every versioning decision starts with.